C++98 到 C++14 面试核心知识点速查手册

本文档专为从 C++98 转型现代 C++ 的开发者编写，聚焦 C++11 和 C++14 中**改变编程范式**的核心特性。面试官通常不会考察生僻的语法，而是侧重考察你对**资源管理、性能优化和并发模型**的理解。

1. 核心基础语法 (Modern Syntax)

这些特性极大地提升了代码的可读性和开发效率，是编写现代 C++ 代码的基础。

- `auto` **与** `decltype` **(类型推导)**
  - **作用**：让编译器在编译期自动推导变量类型。
  - **面试考点**： `auto` 会忽略顶层的 `const` 和引用，如果需要保留引用，需使用 `auto&` 或 `const auto&` 。
- `nullptr` **(空指针常量)**
  - **作用**：取代 C++98 中的宏 `NULL` （通常被定义为整数 `0` ）。
  - **面试考点**：解决函数重载时的歧义问题。例如 `void f(int)` 和 `void f(void*)` ，传入 `NULL` 会调用前者，传入 `nullptr` 会准确调用后者。
- **基于范围的** `for` **循环 (Range-based for loop)**
  - **语法**： `for (const auto& item : container) { ... }`
  - **避坑**：遍历复杂对象时务必使用引用（ `&` ）避免不必要的深拷贝。
- `constexpr` **(常量表达式)**
  - **作用**：指示编译器在**编译期**求值，用于提升运行时性能。相比于 `const` （仅表示运行期只读）， `constexpr` 更加严格。

2. 内存管理：智能指针 (Smart Pointers)

这是 C++ 面试的**绝对核心**。现代 C++ 极度依赖 RAII（资源获取即初始化）机制，要求彻底告别手动 `new` 和 `delete` 。

2.1 `std::unique_ptr` (独占所有权)

- **特点**：同一时刻只能有一个 `unique_ptr` 指向给定对象，离开作用域自动释放内存。**零性能开销**，大小与裸指针相同。
- **面试考点**：
  - **禁用了拷贝构造和拷贝赋值**。
  - 如果需要转移所有权，必须使用 `std::move()` （移动语义）。

2.2 `std::shared_ptr` (共享所有权)

- **特点**：基于**引用计数**。每多一个指针指向对象，计数加 1；指针销毁时计数减 1，减到 0 时释放内存。
- **面试考点 (高频)**：

- **线程安全性**：`shared_ptr` 的引用计数增减是原子操作（线程安全），但对其**指向的对象的读写**不是线程安全的。
- **底层结构**：包含两个指针，一个指向对象，另一个指向控制块（Control Block，存储引用计数等信息）。

2.3 `std::weak_ptr` （弱引用）

- **特点**：配合 `shared_ptr` 使用，**不增加引用计数**。主要用于观察对象，但不控制对象的生命周期。
- **面试考点**：
  - **解决循环引用**：对象 A 包含指向 B 的 `shared_ptr` ，B 包含指向 A 的 `shared_ptr` ，导致计数永远不为 0，内存泄漏。解决办法是将其中一个改为 `weak_ptr` 。
  - **访问方式**：不能直接解引用，必须调用 `lock()` 方法提升为 `shared_ptr` ，判断对象是否还存活再使用。

3. 性能革命：右值引用与移动语义 (Move Semantics)

C++11 性能大幅超越 C++98 的根本原因，也是高级面试必考点。

- **左值与右值**
  - **左值 (Lvalue)**：有名字、能取地址的变量。
  - **右值 (Rvalue)**：临时的、没有名字的、即将销毁的值（如函数返回的临时对象、字面量 `10` ）。
- **右值引用 (`T&&`) 与 `std::move`**
  - `std::move` **不移动任何东西**，它只是一个强制类型转换，将左值无条件转换为右值引用。
  - 它的意义在于触发类的**移动构造函数**或**移动赋值运算符**。
- **面试考点：手写移动构造函数**
  - 核心思想：不要深拷贝（分配新内存+复制数据），而是直接把临时对象的内部指针“偷”过来，然后把临时对象的指针置为 `nullptr` 。

```
class MyString {
    char* data;
public:
    // 移动构造函数
    MyString(MyString&& other) noexcept : data(other.data) {
        other.data = nullptr; // 必须置空，防止析构函数释放这段内存
    }
};
```

- **完美转发 (`std::forward`)**
  - 在模板编程中，当参数被传递给另一个函数时，保持其原始的左值或右值属性不变。

4. 并发与多线程 (Concurrency)

告别平台相关的 `pthread` ，拥抱 C++ 标准库原生的多线程支持。

4.1 线程基础 (`std::thread`)

- **创建与销毁**：创建 `std::thread` 对象即启动线程。必须在线程对象析构前调用 `join()` （阻塞主线程等待子线程结束）或 `detach()` （分离，让子线程在后台运行）。

4.2 互斥锁与 RAII (`std::mutex`)

- **绝对不要**手动调用 `mutex.lock()` 和 `unlock()` ，一旦中间发生异常会导致死锁。
- `std::lock_guard` ：最轻量级的 RAII 锁。构造时加锁，析构（离开作用域）时自动解锁。
- `std::unique_lock` ：功能更强大的锁，支持延迟加锁、手动解锁、结合条件变量使用（代价是比 `lock_guard` 稍微占用多一点点资源）。

4.3 条件变量 (`std::condition_variable`)

- **作用**：线程间的同步与通信。允许一个线程休眠（释放锁），直到被另一个线程唤醒。
- **面试考点**：
  - **必须配合** `std::unique_lock` **使用**。
  - **虚假唤醒 (Spurious Wakeup)**：线程可能在没有被 `notify` 的情况下醒来，因此 `wait` 必须放在 `while` 循环中检查条件，或者使用 `wait` 的 Lambda 重载版本：

```
cv.wait(lock, []{ return !queue.empty(); }); // 只有队列不为空才继续往下走
```

4.4 异步任务 (`std::async` , `std::future`)

- 提供比 `std::thread` 更高级的抽象，能够方便地获取线程的返回值，或捕获子线程抛出的异常。

5. 函数式编程与回调

现代网络框架（如 Drogon、asio）中大量应用。

- **Lambda 表达式 (匿名函数)**
  - **语法**：`[捕获列表](参数列表) -> 返回类型 { 函数体 }`
  - **捕获列表坑点**：`[&]` 按引用捕获所有外部变量。如果 Lambda 被保存下来（如作为异步回调）并在稍后执行，而此时外部变量已销毁，将导致**悬垂引用**崩溃。异步回调中推荐按值捕获 `[=]` 或明确写出捕获的变量。
- `std::function` **与** `std::bind`
  - `std::function` 是一层多态包装器，可以统一存储普通函数、Lambda 表达式、类的成员函数。
  - `std::bind` 用于绑定函数的部分参数，C++14 后建议直接使用 Lambda 替代 `std::bind` 。

6. C++14 重要补全 (加分项)

向面试官展示你不仅懂 C++11，还紧跟后续演进：

1. `std::make_unique` ：C++11 只有 `make_shared` ，C++14 补齐了 `make_unique` 。**最佳实践：永远使用** `make_unique/make_shared` **代替** `new` ，以确保异常安全并提升性能（特别是对于 `shared_ptr` ）。
2. **泛型 Lambda**：允许 Lambda 的参数使用 `auto` 进行类型推导。

   ```
   auto add = [](auto x, auto y) { return x + y; };
   ```

3. **读写锁 (** `std::shared_timed_mutex` **)**：支持共享锁定（多读）和排他锁定（一写）。非常适合读多写少的并发场景。

   面试准备建议： 重点准备“手写基于 `mutex` 和 `condition_variable` 的线程安全队列（生产者-消费者模型）”以及“手写带移动语义的简单自定义类”。